

FASK: Future-Aware Visuomotor Policy Learning via State–Action Coupled Koopman Dynamics

Anonymous Authors

Abstract—Diffusion-based imitation learning has emerged as a powerful paradigm for learning expressive visuomotor policies from demonstrations. However, existing action diffusion policies primarily optimize action likelihood without explicitly evaluating whether generated actions induce task-consistent future state transitions. Consequently, they lack the ability to anticipate the future consequences of generated actions, making it difficult to maintain task progress when future state evolution depends on complex interactions or changing task conditions. To address this issue, we introduce FASK (Future-Aware State–Action Koopman), which couples diffusion-based action generation with state-action bilinear Koopman dynamics to predict the future state consequences of intermediate action proposals before execution. Specifically, FASK learns a state-action coupled Koopman dynamics model to predict how candidate actions influence future latent states, enabling the policy to evaluate both action similarity and task-relevant future consequences. The predicted consequence representation is then used as a self-conditioning signal at the same diffusion noise level to refine the action proposal without introducing an additional diffusion transition. Extensive experiments in both simulation and real-world settings validate the effectiveness of FASK. Project page: <https://silent-node1.github.io/FASK/>.

I. INTRODUCTION

Learning visuomotor policies from demonstrations enables robots to acquire manipulation skills without manually designing task-specific control rules or reward functions. Recent advances include diffusion-based action generation [1], [8], 3D visual representation learning [2], [3], and the incorporation of geometric and kinematic priors [9], [10]. These approaches improve the ability of visuomotor policies to represent multimodal behaviors, capture scene geometry, and exploit structural information for manipulation.

However, improvements in action generation and observation representation do not directly provide feedback about the future consequences of candidate actions. Standard diffusion policies learn to generate action sequences that match expert demonstrations, but do not explicitly predict the future state transitions induced by intermediate action proposals [1], [2]. Geometric equivariance and kinematic structure introduce useful priors for policy learning, yet do not by themselves establish how a particular candidate sequence would change the current robot–scene state [9], [10]. Consequently, a proposal may appear plausible in action space while leading to a future state that hinders task completion, particularly when the effect of an action depends strongly on the current robot–scene state. This motivates incorporating candidate-conditioned consequence prediction into denoising, so that anticipated state evolution can inform action refinement before execution.

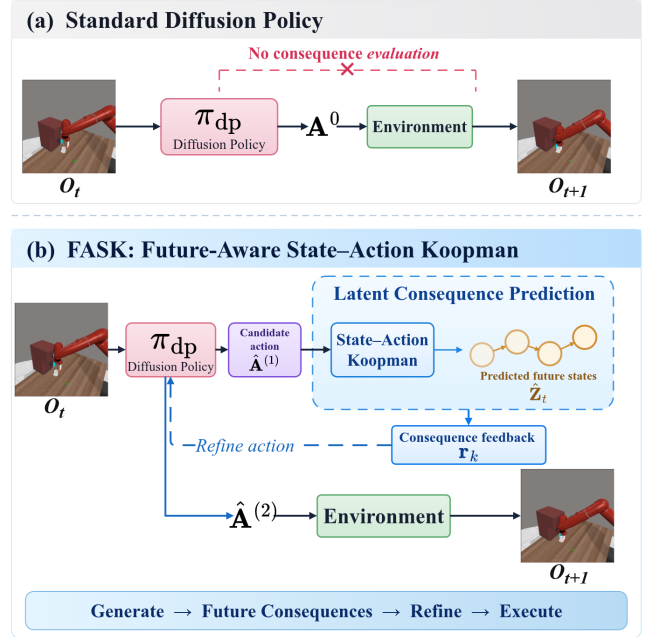


Fig. 1. Comparison between a standard diffusion policy and FASK. (a) A standard diffusion policy generates actions without explicitly evaluating their future consequences. (b) FASK predicts the latent state evolution induced by an intermediate action proposal and feeds this consequence back to refine the action before execution.

To address this problem, we introduce FASK (**Future-Aware State–Action Koopman**), which combines diffusion-based action generation with latent consequence prediction. As illustrated in Fig. 1, FASK treats an intermediate action estimate as a candidate sequence and predicts the short-horizon latent trajectory that it would induce from the current state. This prediction is implemented with a state–action coupled bilinear Koopman consequence model [16], [17]. In addition to acting as a direct control input, each candidate action modulates the latent evolution operator, allowing different candidate sequences to produce distinct predicted consequences. Expert future observations supervise the latent dynamics during training, without requiring reconstruction of future images or point clouds. The predicted consequence is then used to refine the action within the diffusion process. At selected low-noise steps, the first denoiser pass produces a candidate action sequence. The Koopman model predicts its latent consequence, which is provided to a second denoiser pass using the same noisy action sample and the same diffusion timestep. The scheduler advances only after the refined action estimate is obtained, so the refinement does

not introduce an additional noise-level transition.

The contributions of this work are threefold:

- We introduce **FASK**, a future-aware diffusion framework that augments action-space denoising with explicit prediction of the short-horizon latent consequences induced by intermediate action proposals.
- We develop a **consequence-aware refinement mechanism** that combines a state-action coupled bilinear Koopman model for candidate-specific latent rollout with same-noise diffusion refinement.
- We conduct extensive experiments on both simulation benchmarks and real-world robotic manipulation tasks to demonstrate the effectiveness of FASK.

II. RELATED WORK

A. Diffusion Policies for Robot Control

Diffusion models generate samples through iterative denoising [6], [7]. For visuomotor control, action diffusion captures multimodal behaviors and generates temporally coherent action sequences for receding-horizon execution [1], while consistency distillation improves inference efficiency by reducing the number of sampling steps [8]. Other approaches improve spatial generalization through compact point-cloud representations [2], geometric equivariance [9], and graph-based kinematic modeling for bimanual manipulation [10]. Self-conditioning further provides a mechanism for incorporating a model’s previous prediction into a subsequent prediction [11]. These developments improve the representation, structure, and efficiency of action generation or reuse previous estimates during sampling. In contrast, FASK conditions action refinement on the predicted latent consequences of an intermediate action proposal. Rather than directly feeding back the previous action estimate, FASK rolls it out through a learned dynamics model and encodes the resulting future-state trajectory as feedback. The proposal and refinement use the same noisy action sample and diffusion timestep, allowing consequence feedback to influence the action estimate before the scheduler advances.

B. Latent Dynamics and Koopman Models

Learned latent dynamics provide compact predictions for planning and control without requiring pixel-space rollouts [12]. In robot learning, future prediction has also been explored for 3D visual representation learning [3], [20] and structured graph modeling from videos [13]. Recent model-based approaches also explore predictive dynamics for policy learning. LaDi-WM [14] predicts future latent states using a diffusion world model to improve policy learning, while DynBridge [21] connects generated interaction dynamics with a dynamics-guided action predictor. Koopman-based approaches provide a structured formulation of dynamics by seeking observables whose evolution admits a linear representation in a lifted space [15]. For controlled nonlinear systems, bilinear realizations explicitly capture interactions between lifted states and control inputs [16], [17]. In contrast to using future predictions for representation learning, policy conditioning alone, or trajectory optimization, FASK uses

controlled latent rollouts to connect intermediate action proposals with their predicted consequences during denoising. Specifically, a finite-dimensional, low-rank bilinear model allows candidate actions to modulate the latent evolution operator, producing candidate-specific consequence feedback for same-noise action refinement without assuming an exact Koopman-invariant subspace.

III. METHOD

A. Method Overview

Given an L -step observation history $\mathcal{O}_t^{(L)}$ consisting of point clouds and robot proprioceptive states, the policy predicts an H -step action sequence and executes it in a receding-horizon manner. FASK treats an intermediate denoising estimate as a candidate action hypothesis, predicts its short-horizon latent consequence, and feeds this consequence back to refine the action:

$$\hat{\mathbf{A}}^{(1)} \longrightarrow \mathbf{A}^{\text{qry}} \longrightarrow \hat{\mathbf{Z}}_t^{\text{qry}} = F_\phi^{(H_c)}(\mathbf{z}_t, \mathbf{A}^{\text{qry}}) \longrightarrow \mathbf{r}_k. \quad (1)$$

$$(\mathbf{x}_k, k, \mathbf{c}_t, \mathbf{r}_k) \longrightarrow \hat{\mathbf{A}}^{(2)}. \quad (2)$$

Here, $F_\phi^{(H_c)}$ rolls out the current latent state $\mathbf{z}_t$ under the query action sequence $\mathbf{A}^{\text{qry}}$, and $\mathbf{r}_k$ summarizes the resulting latent trajectory without predicting a scalar value or reward. Both $\hat{\mathbf{A}}^{(1)}$ and $\hat{\mathbf{A}}^{(2)}$ are inferred from the same noisy sample $\mathbf{x}_k$ at diffusion step k ; the scheduler advances only after refinement. Demonstrated future transitions supervise consequence modeling during training. Fig. 2 presents the full pipeline.

B. Action-Modulated Koopman Consequence Modeling

At diffusion step k , let $\mathbf{A}^{\text{qry}}$ denote the short action sequence derived from an intermediate denoising estimate. Given the current latent state $\mathbf{z}_t$, FASK predicts the state evolution that this action hypothesis would induce:

$$(\mathbf{z}_t, \mathbf{A}^{\text{qry}}) \longmapsto \hat{\mathbf{Z}}_t^{\text{qry}} = F_\phi^{(H_c)}(\mathbf{z}_t, \mathbf{A}^{\text{qry}}), \quad (3)$$

where $\hat{\mathbf{Z}}_t^{\text{qry}}$ is the predicted H_c -step latent trajectory. Conditioning the rollout on both state and action allows different candidate actions under the same observation to produce different predicted consequences.

1) *Latent Consequence Modeling with Action-Modulated Koopman Dynamics:* We perform future consequence prediction in a compact latent space rather than explicitly reconstructing future point clouds. Given the L -step observation history $\mathcal{O}_t^{(L)}$, we first encode the current robot-scene state as

$$\mathbf{z}_t = E_O(\mathcal{O}_t^{(L)}), \quad (4)$$

where $\mathbf{z}_t \in \mathbb{R}^{d_z}$ captures both the scene geometry and robot proprioceptive state. The corresponding condition used by the diffusion policy is obtained through

$$\mathbf{c}_t = P_C(\mathbf{z}_t), \quad (5)$$

where $P_C(\cdot)$ denotes the conditioning projection. In this way, the latent representation provides a shared state space

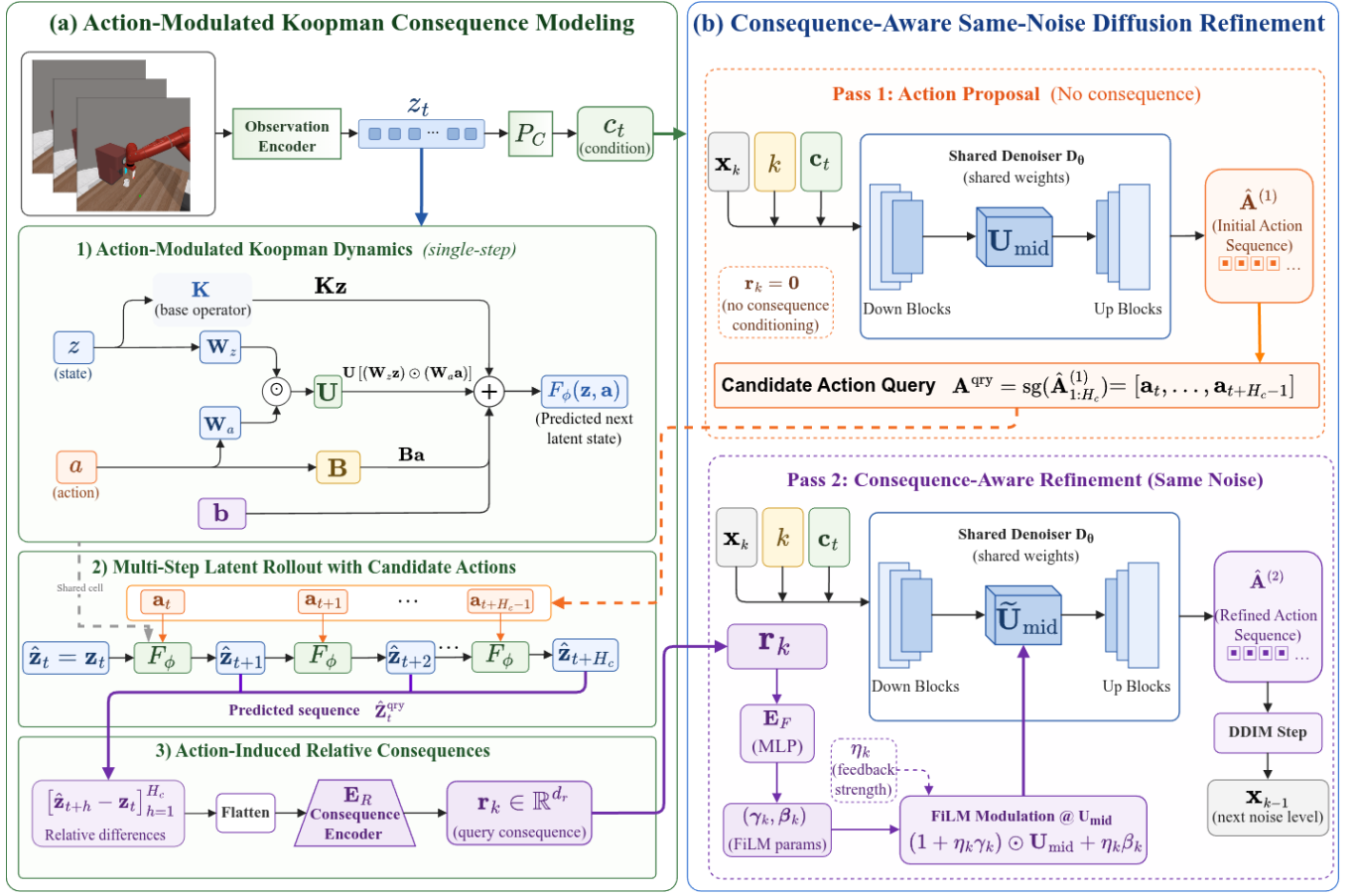


Fig. 2. Overview of FASK. (a) The action-modulated Koopman model rolls out the latent consequence induced by a candidate action sequence. (b) A shared denoiser first proposes the candidate action and then uses its encoded consequence to refine the action at the same diffusion noise level before a single DDIM update.

for both diffusion-based action generation and future consequence prediction.

At diffusion denoising step k , we denote the H_c -step query action sequence used for consequence prediction as

$$A^{qy} = [a_t, \dots, a_{t+H_c-1}], \quad (6)$$

where each $a_{t+h} \in \mathbb{R}^{d_a}$ is treated as the control input for one step of the latent dynamics model. For simplicity, we use $(z, a)_t$ to denote a generic latent state–action pair.

A conventional controlled latent transition typically models the next latent state as the combination of autonomous state evolution and an additive action-dependent control effect, which can be written as

$$F(z, a) = Kz + Ba + b, \quad (7)$$

where K denotes the Koopman operator that characterizes the intrinsic latent-state dynamics, Ba represents the influence of the control action, and b denotes a learnable bias term. Although this formulation incorporates the action through the additive control term Ba , its effect is independent of the current state. Such an assumption is restrictive for robotic manipulation, where the same action command can lead to different state transitions depending on the robot configuration, object pose, and interaction condition. In other

words, the effect of an action should depend on the state under which it is applied.

To capture such state-dependent action effects, we introduce a low-rank action-modulated Koopman transition:

$$F_\phi(z, a) = Kz + Ba + U[(W_z z) \odot (W_a a)] + b, \quad (8)$$

where $\odot$ denotes element-wise multiplication. The multiplicative interaction explicitly couples the latent state and action, allowing the effect of an action to vary with the current robot–scene state. More specifically, the interaction term can be expanded as

$$U[(W_z z) \odot (W_a a)] = \sum_{\ell=1}^r u_\ell (w_{z,\ell}^\top z) (w_{a,\ell}^\top a), \quad (9)$$

where r denotes the interaction rank. The bilinear interaction can be equivalently rewritten in the controlled Koopman form

$$Kz + \sum_{m=1}^{d_a} a_m N_m z + Ba + b, \quad (10)$$

where $N_m = U \text{Diag}(W_a[:, m]) W_z \in \mathbb{R}^{d_z \times d_z}$ denotes the state-transition modulation associated with the m -th action dimension. Therefore, the term $\sum_{m=1}^{d_a} a_m N_m z$ modulates

the latent evolution according to both the current state and the applied action.

Accordingly, the transition can be equivalently interpreted as

$$F_\phi(\mathbf{z}, \mathbf{a}) = \mathbf{K}_\phi(\mathbf{a})\mathbf{z} + \mathbf{B}\mathbf{a} + \mathbf{b}, \quad (11)$$

where

$$\mathbf{K}_\phi(\mathbf{a}) = \mathbf{K} + \sum_{m=1}^{d_a} a_m \mathbf{N}_m. \quad (12)$$

We refer to $\mathbf{K}_\phi(\mathbf{a})$ as the *action-modulated Koopman evolution operator*. Unlike the additive control term, which directly shifts the latent state, $\mathbf{K}_\phi(\mathbf{a})$ changes how the current state evolves under the selected action. Therefore, different candidate actions generated during diffusion denoising instantiate different evolution operators from the same shared operator family, leading to candidate-specific future consequences without requiring an independent dynamics model for each possible action.

2) *Candidate Action Rollout*: Given a query action sequence $\mathbf{A}^{\text{qry}}$ (Eq. (6)) at denoising step k , we evaluate its potential consequence by rolling out the learned latent dynamics. Starting from the current latent state $\hat{\mathbf{z}}_t = \mathbf{z}_t$, the rollout proceeds recursively as

$$\hat{\mathbf{z}}_{t+h+1} = \mathbf{K}_\phi(\mathbf{a}_{t+h})\hat{\mathbf{z}}_{t+h} + \mathbf{B}\mathbf{a}_{t+h} + \mathbf{b}, \quad h = 0, \dots, H_c - 1. \quad (13)$$

The resulting latent trajectory $\hat{\mathbf{z}}_t^{\text{qry}} = [\hat{\mathbf{z}}_{t+1}, \dots, \hat{\mathbf{z}}_{t+H_c}]$ represents the predicted robot-scene evolution induced by the current action hypothesis. Rather than reconstructing future point clouds, we summarize this predicted evolution directly in the latent space. Specifically, the state changes relative to the current latent state are encoded as

$$\mathbf{r}_k = E_R(\text{Flatten}[\hat{\mathbf{z}}_{t+1} - \mathbf{z}_t, \dots, \hat{\mathbf{z}}_{t+H_c} - \mathbf{z}_t]), \quad (14)$$

where $E_R(\cdot)$ denotes the consequence encoder. The resulting feature $\mathbf{r}_k$ compactly describes how the current robot-scene state is expected to evolve under $\mathbf{A}^{\text{qry}}$ and will be used for consequence-aware action refinement in the following section.

To learn a predictive latent dynamics model, we supervise F_ϕ separately using demonstrated state-action transitions $(\mathbf{z}_t, \mathbf{a}_t^*, \mathbf{z}_{t+1}^*)$. Future latent targets are constructed with an exponential-moving-average target encoder $\bar{E}_O$:

$$\mathbf{z}_{t+h}^* = \text{sg} \left[\bar{E}_O \left(\mathcal{O}_{t+h}^{(L)} \right) \right], \quad h = 1, \dots, H_c, \quad (15)$$

where $\text{sg}(\cdot)$ denotes stop-gradient. Let

$$\mathbf{Z}_t^* = [\mathbf{z}_{t+1}^*, \dots, \mathbf{z}_{t+H_c}^*]$$

denote the corresponding demonstrated future latent trajectory, and let

$$\mathbf{A}_c^* = [\mathbf{a}_t^*, \dots, \mathbf{a}_{t+H_c-1}^*]$$

denote the demonstrated action sequence over the same consequence horizon. The dynamics objective is

$$\mathcal{L}_{\text{dyn}} = \left\| F_\phi^{(H_c)}(\mathbf{z}_t, \mathbf{A}_c^*) - \mathbf{Z}_t^* \right\|_2^2. \quad (16)$$

Thus, demonstrated actions are used to supervise the latent dynamics model, whereas candidate actions are used to query the learned dynamics for consequence estimation. The future observations and target encoder are used only during training. At inference time, the learned dynamics model directly evaluates the candidate-specific future evolution induced by an intermediate action hypothesis.

C. Consequence-Aware Same-Noise Diffusion Refinement

To incorporate predicted future consequences into the diffusion process, we perform consequence-aware refinement within selected denoising steps. At diffusion step k , let $\mathbf{x}_k$ denote the current noisy action sequence. FASK first obtains an initial clean-action estimate using the shared denoiser without consequence feedback:

$$\hat{\mathbf{A}}^{(1)} = D_\theta(\mathbf{x}_k, k, \mathbf{c}_t, \mathbf{0}), \quad (17)$$

where $\mathbf{c}_t$ denotes the observation condition defined in Eq. (5). Rather than immediately advancing the diffusion process, we interpret $\hat{\mathbf{A}}^{(1)}$ as a hypothetical action proposal whose future consequence can be evaluated.

The first H_c actions form the query sequence

$$\mathbf{A}^{\text{qry}} = \text{sg} \left(\hat{\mathbf{A}}_{1:H_c}^{(1)} \right). \quad (18)$$

Using the action-modulated Koopman dynamics F_ϕ introduced in Sec. III-B, this candidate action is rolled out from the current latent state $\mathbf{z}_t$, producing a predicted future latent trajectory and its consequence representation $\mathbf{r}_k$ according to Eqs. (13)–(14). Thus, $\mathbf{r}_k$ summarizes how the current robot-scene state is expected to evolve if the proposed action sequence were executed.

We inject this predicted consequence into the bottleneck feature of the shared denoiser through FiLM [19]. Specifically,

$$\begin{aligned} [\gamma_k, \beta_k] &= E_F(\mathbf{r}_k), \\ \tilde{\mathbf{U}}_{\text{mid}} &= (1 + \eta_k \gamma_k) \odot \mathbf{U}_{\text{mid}} + \eta_k \beta_k, \end{aligned} \quad (19)$$

where E_F is an MLP that maps $\mathbf{r}_k$ to the FiLM parameters, and $\eta_k \in [0, 1]$ controls the strength of consequence feedback. We apply refinement only at a subset of low-noise denoising steps, denoted by $\mathcal{K}_{\text{ref}}$, where the action hypothesis is sufficiently structured for meaningful consequence evaluation.

Conditioned on the predicted consequence, the same denoiser then revisits the same noisy action sequence at the same diffusion step:

$$\hat{\mathbf{A}}^{(2)} = D_\theta(\mathbf{x}_k, k, \mathbf{c}_t, \mathbf{r}_k; \eta_k). \quad (20)$$

Importantly, the two passes operate on exactly the same $(\mathbf{x}_k, k)$. The first pass proposes a hypothetical action, the learned dynamics evaluates its expected future consequence, and the second pass uses this consequence to refine the action before the diffusion scheduler changes the noise level. FASK therefore introduces future-consequence reasoning *within* a diffusion step rather than adding an additional denoising transition.

a) Training: To stabilize consequence-aware refinement when first-pass proposals are still immature, we gradually replace demonstrated query actions with model-generated proposals using sequence-level scheduled sampling [18]. Let $\mathbf{A}_c^* = \mathbf{A}_{1:H_c}^*$ and $\hat{\mathbf{A}}_c^{(j)} = \hat{\mathbf{A}}_{1:H_c}^{(j)}$ denote the consequence-horizon action segments. The query sequence is constructed as

$$\mathbf{A}_i^{\text{qry}} = b_i \mathbf{A}_{c,i}^* + (1 - b_i) \text{sg} \left(\hat{\mathbf{A}}_{c,i}^{(1)} \right). \quad (21)$$

Here $b_i \sim \text{Bernoulli}(\rho_s)$, and the expert-query probability ρ_s gradually decreases from one to zero. The refinement branch therefore transitions from demonstrated action queries to the policy’s own first-pass proposals.

Refinement is applied only at selected low-noise steps, with training-time feedback strength

$$\eta_{s,k} = (1 - \rho_s) \mathbb{I}[k \in \mathcal{K}_{\text{ref}}]. \quad (22)$$

Thus, consequence feedback is gradually activated as training shifts toward model-generated queries. We use $\eta_k = \eta_{s,k}$ in the refinement pass.

When consequence-aware refinement is active, both denoising estimates are supervised over the complete action horizon:

$$\mathcal{L}_{\text{diff}} = \frac{1}{2} \sum_{j=1}^2 \ell_{\text{diff}} \left(\hat{\mathbf{A}}^{(j)}, \mathbf{A}^* \right). \quad (23)$$

Here ℓ_{diff} denotes the standard action-diffusion denoising loss. This objective maintains consistency with demonstrated actions for both the initial proposal and the consequence-refined estimate.

Beyond action imitation, we explicitly supervise whether the second pass induces a more task-consistent future. Using frozen dynamics parameters, we measure

$$d_j = \left\| F_{\text{sg}(\phi)}^{(H_c)} \left(\text{sg}(\mathbf{z}_t), \hat{\mathbf{A}}_c^{(j)} \right) - \mathbf{Z}_t^* \right\|_2^2, \quad j \in \{1, 2\}. \quad (24)$$

The consequence refinement objective is

$$\mathcal{L}_{\text{con}} = d_2 + \lambda_r \max(0, d_2 - \text{sg}(d_1)). \quad (25)$$

The first term encourages the refined action to induce a latent future consistent with the demonstrated transition, while the second penalizes refinement when its predicted consequence is worse than that of the initial proposal. This objective is evaluated only for samples at active refinement steps. The query rollout is detached from the refinement objective, and the dynamics parameters are frozen when computing d_1 and d_2 , so that $\mathcal{L}_{\text{con}}$ optimizes the denoiser without altering the learned dynamics.

Combining action imitation, latent dynamics prediction, and consequence-aware refinement, the overall training objective is

$$\mathcal{L} = \mathcal{L}_{\text{diff}} + \lambda_{\text{dyn}} \mathcal{L}_{\text{dyn}} + \lambda_{\text{con}} \mathcal{L}_{\text{con}}. \quad (26)$$

b) Inference: At inference time, the future-target branch and all training losses are removed. High-noise denoising steps directly use the first-pass estimate, while selected low-noise steps in $\mathcal{K}_{\text{ref}}$ apply consequence-aware refinement. For active refinement steps, the query action is obtained from the detached first-pass proposal and the feedback strength is set to $\eta_k = 1$. Inactive steps use $\hat{\mathbf{A}}^{(1)}$, whereas active steps use $\hat{\mathbf{A}}^{(2)}$. In both cases, the diffusion scheduler advances only once:

$$\mathbf{x}_{k-1} = \text{DDIMStep} \left(\mathbf{x}_k, \hat{\mathbf{A}}_k, k \right), \quad (27)$$

where $\hat{\mathbf{A}}_k$ denotes the action estimate selected at the current step. After completing reverse diffusion, the policy executes the next N_{act} actions and replans from the latest observation history.

IV. EXPERIMENTS

A. Experimental Setup

Benchmarks. We evaluate FASK on MetaWorld [4] and Adroit [5], which cover arm-based manipulation and high-DoF dexterous-hand control. We use 14 MetaWorld tasks with 25 expert demonstrations per task and the Adroit *Door* and *Pen* tasks with 100 demonstrations per task.

Baselines. We compare with Diffusion Policy [1], DP3 [2], FVP [3], and AFRO [20]. Diffusion Policy and DP3 provide image- and point-cloud-based action diffusion baselines, respectively, while FVP and AFRO incorporate future prediction or dynamics-aware visual representation learning.

Implementation Details. FASK is trained end-to-end from scratch for 1000 epochs using AdamW with $(\beta_1, \beta_2) = (0.95, 0.999)$, a learning rate of 1×10^{-4} with cosine annealing, and a batch size of 128. The policy uses two observation steps, predicts a 16-step action sequence, and executes eight actions before replanning. We set the consequence horizon to $H_c = 8$, latent dimension to $d_z = 128$, and interaction rank to $r = 8$. Consequence feedback is applied at the four lowest-noise steps of a 10-step DDIM sampler. The probability ρ_s is cosine-decayed from 1 to 0 over the first 30% of training, with $(\lambda_{\text{dyn}}, \lambda_{\text{con}}, \lambda_r) = (0.1, 0.02, 0.5)$ and a target-encoder EMA decay of 0.995. For simulated tasks, we evaluate every 100 epochs using 50 rollouts and report results from the best-performing checkpoint. All baselines use the same demonstrations and evaluation protocol. All experiments are conducted on a single NVIDIA RTX 4090 GPU.

B. Quantitative Results

Table I summarizes the main results. On MetaWorld, FASK achieves the best or tied-best performance on 10 of the 14 tasks and obtains the highest average success rate in every difficulty group. Its overall mean success rate reaches 79.3%, compared with 76.0% for AFRO, 69.7% for DP3, 54.3% for FVP, and 38.4% for Diffusion Policy. FASK remains particularly effective on the Medium and Hard task groups, where it achieves average success rates of 74.7% and 60.0%, respectively, compared with 69.7% and 55.0% for the

TABLE I
SUCCESS RATES (%) ON 14 METAWORD TASKS AND TWO ADROIT TASKS. BEST RESULTS ARE SHOWN IN BOLD.

Method	Easy			Medium					
	Dial Turn	Handle Press	Peg Unplug Side	Bin Picking	Coffee Pull	Peg Insert Side	Push Wall	Soccer	Sweep
Diffusion Policy [1]	60	82	78	16	34	34	56	22	32
DP3 [2]	76	86	86	18	90	74	78	38	92
FVP [3]	72	88	80	16	66	70	36	34	44
AFRO [20]	78	98	88	20	92	92	80	36	98
FASK (Ours)	70	96	100	30	100	90	88	40	100

Method	Hard		Very Hard			MetaWorld		Adroit	
	Pick Out of Hole	Push	Stick Pull	Stick Push	Pick Place Wall	Mean	S.R.	Door	Pen
Diffusion Policy [1]	0	30	20	66	8	38.4		40	16
DP3 [2]	24	74	58	88	94	69.7		70	80
FVP [3]	26	42	24	84	78	54.3		66	76
AFRO [20]	32	78	78	100	94	76.0		82	84
FASK (Ours)	34	86	76	100	100	79.3		80	88

strongest competing method, AFRO. These results show that FASK improves over both standard action diffusion policies and methods that incorporate future prediction or dynamics-aware visual representations.

On Adroit, FASK achieves 80% success on *Door* and the best result of 88% on *Pen*, yielding the highest mean success rate of 84.0%. The corresponding mean success rates of AFRO, DP3, FVP, and Diffusion Policy are 83.0%, 75.0%, 71.0%, and 28.0%, respectively. Together with the MetaWorld results, this demonstrates that future-aware action refinement benefits both arm-based manipulation and high-dimensional dexterous-hand control.

C. Ablation Studies

In this section, we evaluate the effects of future feedback, state-action coupling, and consequence supervision on four MetaWorld tasks spanning different difficulty levels, namely *Handle Press*, *Push Wall*, *Push*, and *Stick Pull*, as well as Adroit *Pen*. All variants use the same demonstrations, training budget, and evaluation protocol.

TABLE II
ABLATION RESULTS ON FOUR METAWORD TASKS AND THE ADROIT PEN TASK. SUCCESS RATES ARE REPORTED IN %, WITH BEST RESULTS IN BOLD.

Variant	Handle Press	Push Wall	Push	Stick Pull	Pen	Avg.
Feedback disabled	84	78	76	70	86	78.8
Linear dynamics	90	86	84	68	88	83.2
w/o $\mathcal{L}_{\text{con}}$	90	84	82	70	80	81.2
Full FASK	96	88	86	76	88	86.8

As shown in Table II, consequence feedback provides the denoiser with candidate-specific future information during action refinement. Disabling this feedback only at inference lowers performance on every task, showing that the predicted consequence actively contributes to action generation rather

than serving only as an auxiliary training signal. The bi-linear interaction models how the effect of an action varies with the current latent state. Replacing it with the additive transition $F_\phi(z, a) = Kz + Ba + b$ degrades four of the five tasks, with the largest change on *Stick Pull* (68% vs. 76%), supporting the need for state-dependent action effects when evaluating candidate consequences. Finally, $\mathcal{L}_{\text{con}}$ explicitly trains the second-pass action to induce a future closer to the demonstrated transition. Removing this objective decreases the average success rate from 86.8% to 81.2%, indicating that predicting and feeding back consequences alone does not provide the same benefit as directly supervising the refinement direction. Together, these results support the complete mechanism: the dynamics model predicts candidate-specific consequences, feedback uses them to refine the action, and consequence supervision guides that refinement toward the demonstrated future.

D. Analysis

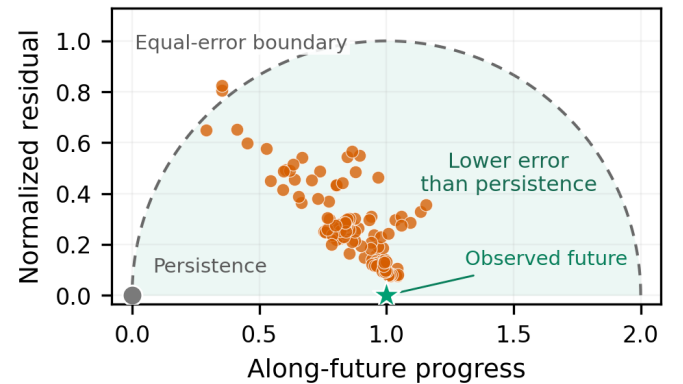


Fig. 3. Target-aligned latent prediction on held-out MetaWorld *Push* transitions. Points inside the dashed equal-error boundary have lower error than persistence.

Latent consequence prediction. Since consequence-

aware refinement relies on the predicted latent future, we first examine whether F_ϕ captures non-trivial state evolution. Under demonstrated-action rollouts, 117 of 119 predictions (98%) are closer to the observed future than the persistence baseline, with a median relative error of $0.31\times$ (Fig. 3). The learned dynamics also maintain lower error throughout the eight-step horizon (Fig. 4), supporting their use for short-horizon consequence estimation.

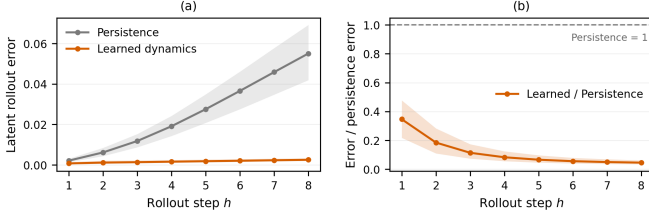


Fig. 4. Multi-step latent dynamics on held-out MetaWorld *Push* transitions. (a) Absolute rollout error; (b) error relative to persistence. Shading denotes bootstrap 95% confidence intervals.

Consequence-guided action correction. To verify that consequence feedback provides meaningful guidance for action generation, we measure whether the refined actions lead to improved future state alignment. Fig. 5 shows that same-noise refinement reduces the future error for 79.8% of the proposals, indicating that consequence feedback consistently steers the refined actions toward the demonstrated future.

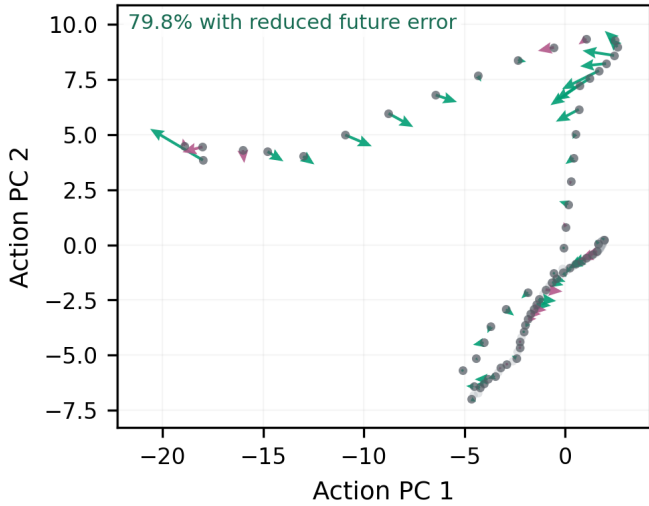


Fig. 5. Same-noise action refinement in a shared PCA space on MetaWorld *Push*. Arrows point from first-pass to refined actions; green and magenta denote decreased and increased error to the observed future, respectively. Arrows are magnified by $35\times$.

State-action interaction. Beyond the effectiveness of refinement, we further investigate how the learned state-action dynamics respond to different motion regimes. As shown in Fig. 6, the relative contribution of the bilinear interaction term increases with observed latent motion, achieving a Pearson correlation of 0.91. This indicates that the learned dynamics adaptively emphasize state-action coupling when larger state transitions occur.

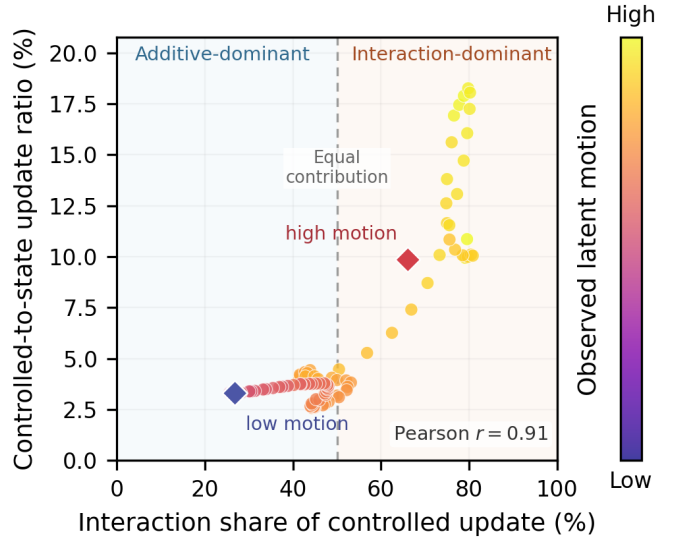


Fig. 6. State-action interaction on MetaWorld *Push*. Color denotes observed latent motion; diamonds mark the low- and high-motion quintile centers.

E. Real-World Experiments

1) *Experimental Setup:* We conduct real-world experiments using a UR5 robotic arm equipped with a parallel gripper and an Intel RealSense D435 RGB-D camera. The robotic platform, workspace, and objects used in our experiments are illustrated in Fig. 7. We evaluate three manipulation tasks with different object geometries and motion requirements: *Potato into Bowl*, which requires grasping an irregularly shaped object and placing it into a target bowl; *Drawer Opening*, which requires grasping a handle and pulling the drawer along its constrained motion direction; and *Block Stacking*, which requires placing a movable block stably on a target block.



Fig. 7. Real-world experimental setup with a UR5 arm, a parallel gripper, an Intel RealSense D435 camera, and the objects used in our tasks.

For each task, we manually collect 40 expert demonstrations through keyboard teleoperation, with the UR5 arm and gripper remotely controlled by the operator. All methods are evaluated over 20 real-world trials per task, and task success

rate is reported as the primary metric.

2) *Task Performance*: Table III summarizes the real-world success rates, while representative execution sequences are shown in Fig. 8. FASK achieves success rates of 85%, 75%, and 75% on *Potato into Bowl*, *Drawer Opening*, and *Block Stacking*, respectively, compared with 70%, 60%, and 55% for DP3. This raises the average success rate from 61.7% to 78.3%, with consistent improvements across all three tasks. The qualitative sequences illustrate how FASK completes the corresponding grasping, transport, articulated motion, and placement stages. Together, these results demonstrate that consequence-aware refinement transfers effectively to real-world manipulation.

TABLE III

REAL-WORLD SUCCESS RATES (%) OVER TWENTY TRIALS PER TASK.

Method	Potato into Bowl	Drawer Opening	Block Stacking	Avg.
DP3	70	60	55	61.7
FASK (Ours)	85	75	75	78.3

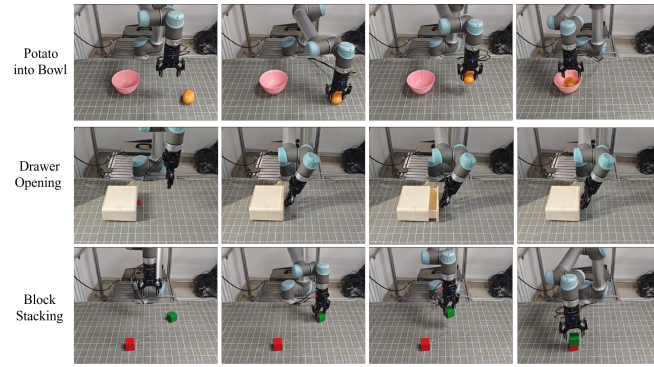


Fig. 8. Representative FASK execution sequences. From top to bottom: *Potato into Bowl*, *Drawer Opening*, and *Block Stacking*; columns show successive stages of each task.

V. CONCLUSION

We presented FASK, a future-aware framework that augments action-space diffusion with predicted state evolution. FASK treats an intermediate action estimate as a hypothetical control sequence and uses its short-horizon latent consequence to refine the action under the same noise condition. Low-rank state-action coupled bilinear Koopman dynamics capture state-dependent control effects, while future-consequence supervision directs refinement toward demonstrated state evolution. The complete model is learned from offline demonstrations without a pretrained policy, additional interaction, or high-dimensional future observation prediction. More broadly, FASK frames future evolution as a criterion within action generation rather than as a separate planning stage.

REFERENCES

[1] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Proc. Robotics: Science and Systems (RSS)*, 2023.

[2] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations,” in *Proc. Robotics: Science and Systems (RSS)*, 2024.

[3] C. Hou, Y. Ze, Y. Fu, Z. Gao, S. Hu, Y. Yu, S. Zhang, and H. Xu, “4D visual pre-training for robot learning,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, pp. 8451–8461, 2025.

[4] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, “Meta-World: A benchmark and evaluation for multi-task and meta reinforcement learning,” in *Proc. Conf. Robot Learning (CoRL)*, pp. 1094–1100, 2020.

[5] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine, “Learning complex dexterous manipulation with deep reinforcement learning and demonstrations,” in *Proc. Robotics: Science and Systems (RSS)*, 2018.

[6] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, 2020.

[7] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[8] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg, “Consistency policy: Accelerated visuomotor policies via consistency distillation,” in *Proc. Robotics: Science and Systems (RSS)*, 2024.

[9] J. Yang, Z. Cao, C. Deng, R. Antonova, S. Song, and J. Bohg, “EquiBot: SIM(3)-equivariant diffusion policy for generalizable and data efficient learning,” in *Proc. Conf. Robot Learning (CoRL)*, pp. 1048–1068, 2025.

[10] Q. Lv, H. Li, X. Deng, R. Shao, Y. Li, J. Hao, L. Gao, M. Y. Wang, and L. Nie, “Spatial-temporal graph diffusion policy with kinematic modeling for bimanual robotic manipulation,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 17394–17404, 2025.

[11] T. Chen, R. Zhang, and G. Hinton, “Analog bits: Generating discrete data using diffusion models with self-conditioning,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[12] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, “Learning latent dynamics for planning from pixels,” in *Proc. Int. Conf. Machine Learning (ICML)*, pp. 2555–2565, 2019.

[13] G. Chen, T. Cui, M. Wang, C. Yang, M. Hu, H. Lu, Y. Mu, Z. Peng, T. Zhou, X. Jiang, Y. Yang, and Y. Yue, “GraphMimic: Graph-to-Graphs generative modeling from videos for policy learning,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 1756–1768, 2025.

[14] Y. Huang, J. Zhang, S. Zou, X. Liu, R. Hu, and K. Xu, “LaDi-WM: A latent diffusion-based world model for predictive manipulation,” in *Proc. Conf. Robot Learning (CoRL)*, pp. 1726–1743, 2025.

[15] B. O. Koopman, “Hamiltonian systems and transformation in Hilbert space,” *Proc. National Academy of Sciences*, vol. 17, no. 5, pp. 315–318, 1931.

[16] D. Bruder, X. Fu, and R. Vasudevan, “Advantages of bilinear Koopman realizations for the modeling and control of systems with unknown dynamics,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4369–4376, 2021.

[17] W. Zhang and J.-S. Li, “Koopman bilinearization of nonlinear control systems,” *arXiv preprint arXiv:2211.07112*, 2022.

[18] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, “Scheduled sampling for sequence prediction with recurrent neural networks,” in *Advances in Neural Information Processing Systems*, 2015.

[19] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *Proc. AAAI Conf. Artificial Intelligence*, 2018.

[20] Q. Liang, B. Cai, M. Lai, S. Zhuang, T. Lin, Y. Qin, Y. Ye, J. Liang, and R. Xu, “Bootstrap dynamic-aware 3D visual representation for scalable robot learning,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 13419–13429, 2026.

[21] A. Wang, Z. Dong, Q. Bai, C. Zhang, Y. Yi, G. Dai, Y. Liu, and M. Wang, “DynBridge: Bridging imagination and control through interaction dynamics for robot manipulation,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 22423–22432, 2026.